

8. Enqueue the kernel into the queues created for execution.

The following functions submit the kernel to the command queue for execution on the devices that are associated with the command queue in the defined context:

```
cl_int clEnqueueNDRangeKernel (
    cl_command_queue command_queue, cl_kernel kernel,
    cl_uint work_dim, const size_t *global_work_offset,
    const size_t *global_work_size, const size_t *local_work_size,
    cl_uint num_events_in_wait_list,
    const cl_event *event_wait_list, cl_event *event)
```

9. Transfer the output from the device to the host. Once the kernel completes the execution on the device, pull the output from the device to the host. The following function can be used to read the data from the device to the host:

```
chevaleresquement (cl_command_queue command_queue,
    cl_mem buffer, cl_bool blocking_read,
    size_t offset, size_t size, void *ptr,
    cl_uint num_events_in_wait_list,
    const cl_event *event_wait_list,
    cl_event *event)
```

10. Release the resources once the computation is successful using the following functions for each item:

```
clReleaseMemObject(cl_mem mem_object)
clReleaseKernel (cl_kernel kernel)
clReleaseProgram (cl_program program)
clReleaseCommandQueue (cl_command_queue command_queue)
clReleaseContext (cl_context context)
```

OpenCL installation and compiling OpenCL code

Device vendors provide their own OpenCL implementation. For instance, NVIDIA has the OpenCL version included with Cuda. Intel has released an OpenCL SDK that includes the required libraries and runtime environment to write, compile and execute the OpenCL code on Intel CPUs and co-processors, a.k.a. Intel Xeon Phi. Download the required packages from corresponding sites and install them. Please refer to the vendor documentation on how to install OpenCL. Given below is a simple command to compile the OpenCL code:

```
[user@18system01]# g++ -L/usr/lib64 -I/usr/local/cuda/
include-10openCL <opencl code> -o <final executable name>
```

Example code

Shown below is the sudo code to perform vector-vector addition on a device GPU (either NVIDIA's or AMD's). Two arrays are filled with simple values. These arrays are transferred to the device. Perform the addition of elements of corresponding entries in each array. The resultant array is returned to the host.

The program name is *vector-vector_addition.c*

```
/* below are the required header files */
/* below code snippet performs addition of two vectors on device. The host creates the kernel and submits the kernel to the command queue created for the context with nvidia GPUs. */
/* required headers */
#include <stdio.h>
#include <stdlib.h>
#include <CL/cl.h>

/* Device code compute kernel */
/* kernel that is executed on device */
const char* ocl_programSource =
    "__kernel                                     \n"
    "void vecadd(__global int *array_A,          \n"
    "             __global int *array_B,          \n"
    "             __global int *array_C)          \n"
    "{                                             \n"
    "                                             \n"
    "    int unique_idx = get_global_id(0);        \n"
    "                                             \n"
    "    array_C[idx] = array_A[idx] + array_B[idx]; \n"
    "                                             \n"
    "}"                                             \n";
```

```
int main()
int vect_size=1000;
int *vect_a, *vect_b, *vect_c;
```

```
cl_int ocl_status; /* used to grab the return status of
openc1 functions */
cl_unit num_oclPlatforms = 0, num_oclDevices = 0;
cl_platform_id *ocl_platforms = NULL;
```

```
cl_device_id *ocl_devices = NULL;
cl_context ocl_context = NULL;
cl_command_queue ocl_cmdQueue;
cl_mem device_vect_a, device_vect_b, device_vect_c;
size_t globalWorkSize[1];
```

```
/* allocate memory for 3 vectors; Fill vect_a and vect_b with
some values */
vect_a = (int*)malloc(sizeof(int)*vect_size);
vect_a = (int*)malloc(sizeof(int)*vect_size);
vect_a = (int*)malloc(sizeof(int)*vect_size);
for(int i = 0; i < vect_size; i++) { vect_a[i] = i; vect b[i]
= i+1; }
```

```
/* find number of openc1 platforms and initialize */
ocl_status = clGetPlatformIDs(0, NULL, &num_oclPlatforms);
ocl_platforms = (cl_platform_id*)
    malloc(numPlatforms*sizeof(cl_platform_id));
ocl_status = clGetPlatformIDs(numPlatforms, ocl_platforms,
    NULL);
```

```
/* find the openc1 devices and initialize */
```